Credit Card Fraud Detection: Technical Report

Oct 29, 2026

Mozammel Hoque Sagor · Machine Learning, Chapter 14 Capstone Project (Project 2: binary classification)

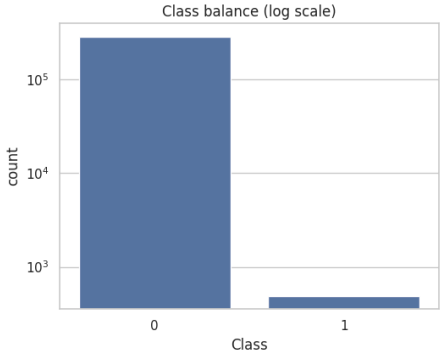
Abstract

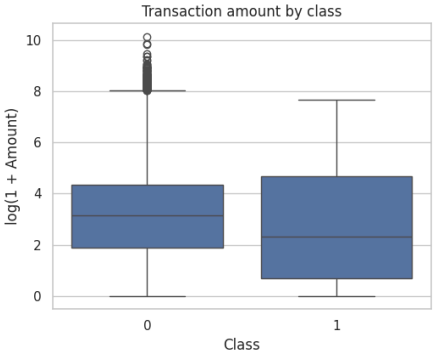
A tuned XGBoost model catches 72 of the 95 fraudulent transactions in a held-out test set and raises only 1 false alarm among 56,651 legitimate transactions (precision 0.986, recall 0.758, F1 0.857, PR-AUC 0.817). The data is the public Kaggle Credit Card Fraud dataset: 284,807 transactions, of which 0.17% are fraud. I compared XGBoost with a Dummy classifier and two Logistic Regression baselines (class weights and SMOTE). The baselines catch more fraud (83 of 95) but flag about 1,400 legitimate transactions to do so, roughly 17 false alarms for every fraud caught, which a bank could not review in practice.

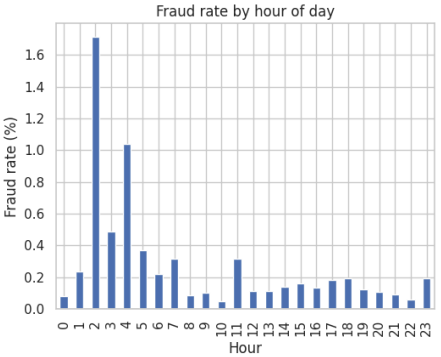
Data exploration

The dataset has 284,807 rows and 31 columns: `Time` (seconds since the first transaction), `V1` to `V28` (anonymised PCA components), `Amount` and the label `Class` (1 = fraud). There are no missing values. I removed 1,081 duplicate rows before splitting, which left 283,726 rows, and split them 80/20 with stratification so both parts keep the same fraud share. The 492 frauds became 473 after deduplication (378 in training plus 95 in test), because 19 duplicated fraud rows were removed.

Split	Transactions	Fraud	Fraud share
Train	226,980	378	0.167%
Test	56,746	95	0.167%







What the exploration showed:

1. **Extreme imbalance.** Only 492 of 284,807 transactions (0.173%) are fraud. A model that always answers "not fraud" scores 99.83% accuracy with 0 recall, so accuracy is not used to judge the models. Precision, recall, F1 and PR-AUC are used instead.
2. **Amount alone does not separate the classes.** On the log scale the median fraud amount is visibly lower than the median legitimate amount, but the two distributions overlap heavily.
3. **Fraud rate varies by hour.** The rate peaks at hour 2 (about 1.7%) and hour 4 (about 1.0%) and is mostly well below 0.5% at other hours. This is only suggestive: the data covers about two days, and few transactions happen at night, so the night-time rates are noisy.
4. **Some anonymised features track fraud.** `V17`, `V14` and `V12` have the strongest negative correlation with fraud (roughly -0.33, -0.30 and -0.26, read from the notebook chart), while `V11` and `V4` are positively correlated (roughly +0.15 and +0.13). Because the features are anonymised, I cannot say what they mean.

Methodology

I built the pipeline so that nothing from the test set can influence training.

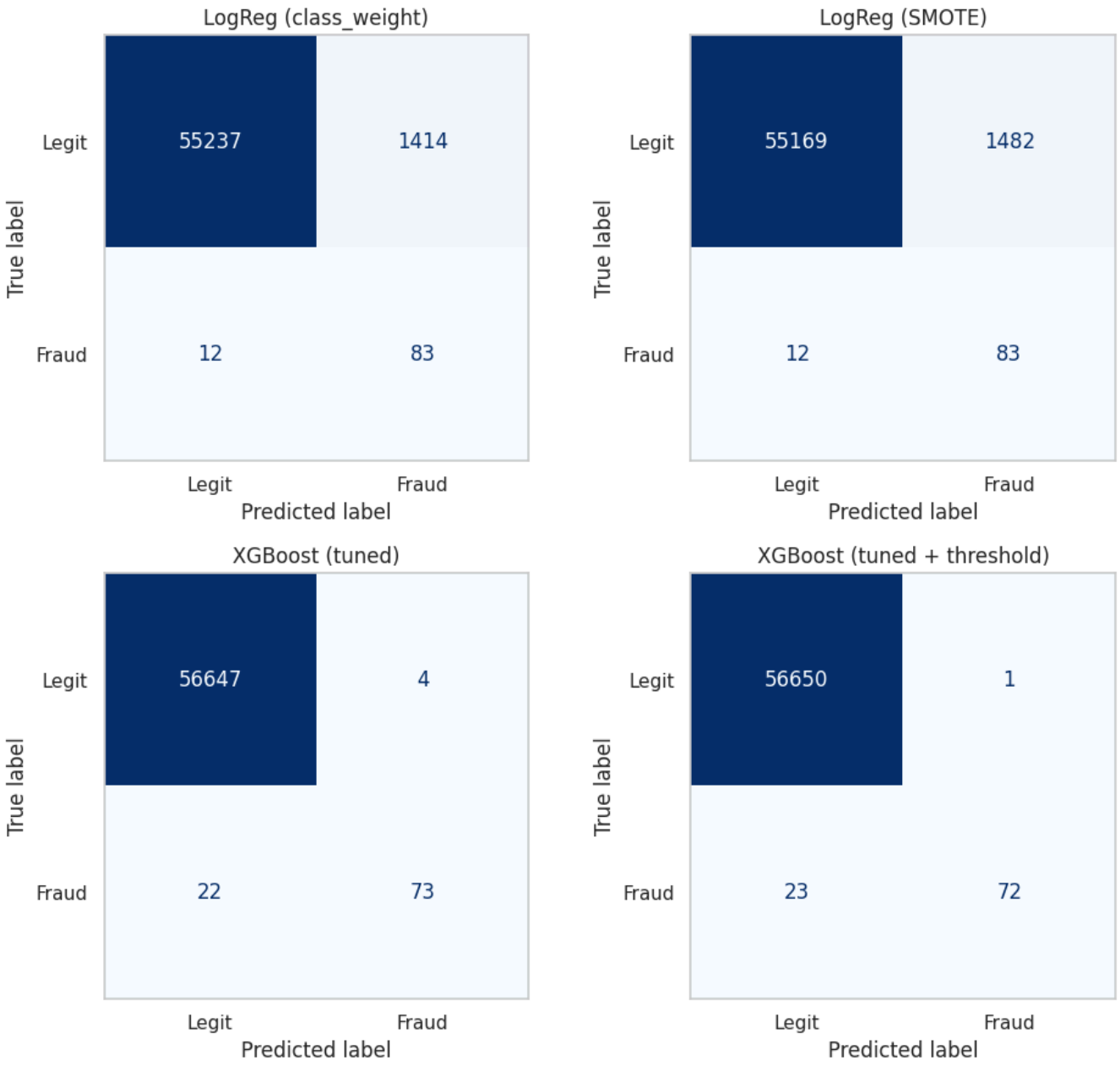
- **Preprocessing.** Duplicates were removed before the split. The raw `Time` column was replaced by `Hour` (hour of the 24-hour cycle). `Amount` and `Hour` were scaled with `RobustScaler`, chosen because `Amount` has large outliers; `V1` to `V28` are already PCA-transformed and were passed through. The scaler sits inside each model pipeline, so it is fitted on training data only.
- **Baselines.** A Dummy classifier (always "not fraud") shows why accuracy misleads. Logistic Regression was trained two ways: with `class_weight="balanced"`, and with SMOTE, which creates synthetic fraud examples. SMOTE runs inside an `imblearn` pipeline, so it only touches training data.
- **Complex model.** XGBoost builds many small trees in sequence, each correcting the errors of the previous ones, and handles non-linear patterns that Logistic Regression cannot. I chose it because it is a common strong choice for tabular data, and its scale_pos_weight setting lets it give more weight to the rare fraud class.
- **Hyperparameter tuning.** `RandomizedSearchCV` tried 10 parameter sets with 3-fold stratified cross-validation on the training data, scoring PR-AUC (`average_precision`), which suits rare-event problems. The best set was `n_estimators` 500, `max_depth` 8, `learning_rate` 0.1, `subsample` 0.7, `colsample_bytree` 1.0, `min_child_weight` 1 and `scale_pos_weight` about 599.5 (the ratio of legitimate to fraud transactions in the training set). Its cross-validated PR-AUC was 0.854.

- **Decision threshold.** A transaction is flagged when its fraud probability is at least a threshold. I chose 0.840, the value that maximised F1 (0.866) on out-of-fold training predictions, so the test set played no part in the choice.
- **Metrics.** Precision (of the transactions flagged, how many are fraud), recall (of all frauds, how many are caught), F1 (their balance) and PR-AUC (quality of the fraud scores across all thresholds).

Results

The tuned XGBoost with the chosen threshold is the best model: it keeps precision at 0.986 and recall at 0.758, for the highest F1 (0.857). All numbers below are on the 56,746-transaction test set, which contains 95 frauds.

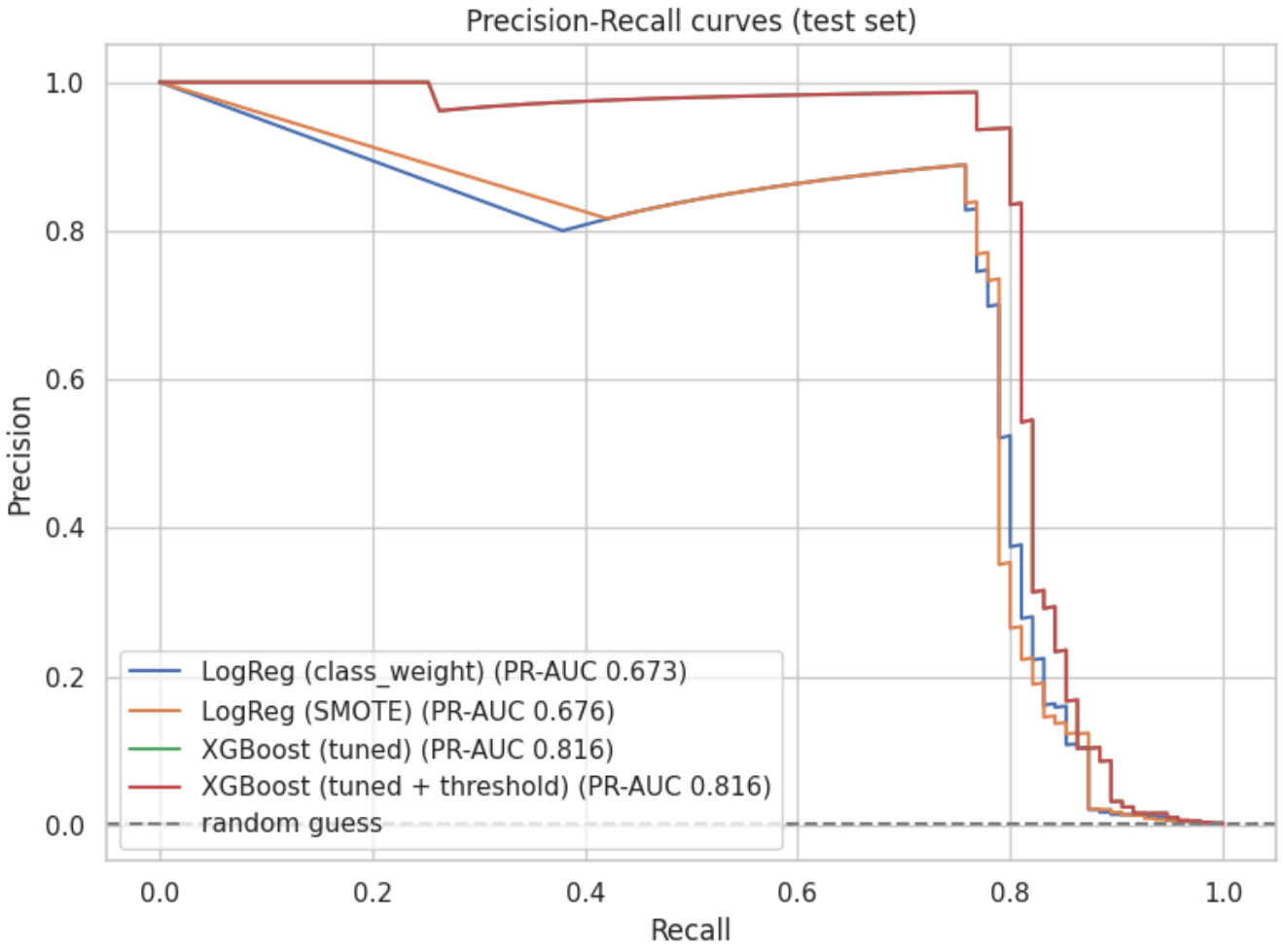
Model	Frauds caught (of 95)	False alarms (of 56,651 legitimate)	Precision	Recall	F1	PR-AUC
Dummy (always not fraud)	0	0	0.000	0.000	0.000	0.002
Logistic Regression (class weights)	83	1,414	0.055	0.874	0.104	0.673
Logistic Regression (SMOTE)	83	1,482	0.053	0.874	0.100	0.676
XGBoost (tuned)	73	4	0.948	0.768	0.849	0.817
XGBoost (tuned + threshold)	72	1	0.986	0.758	0.857	0.817



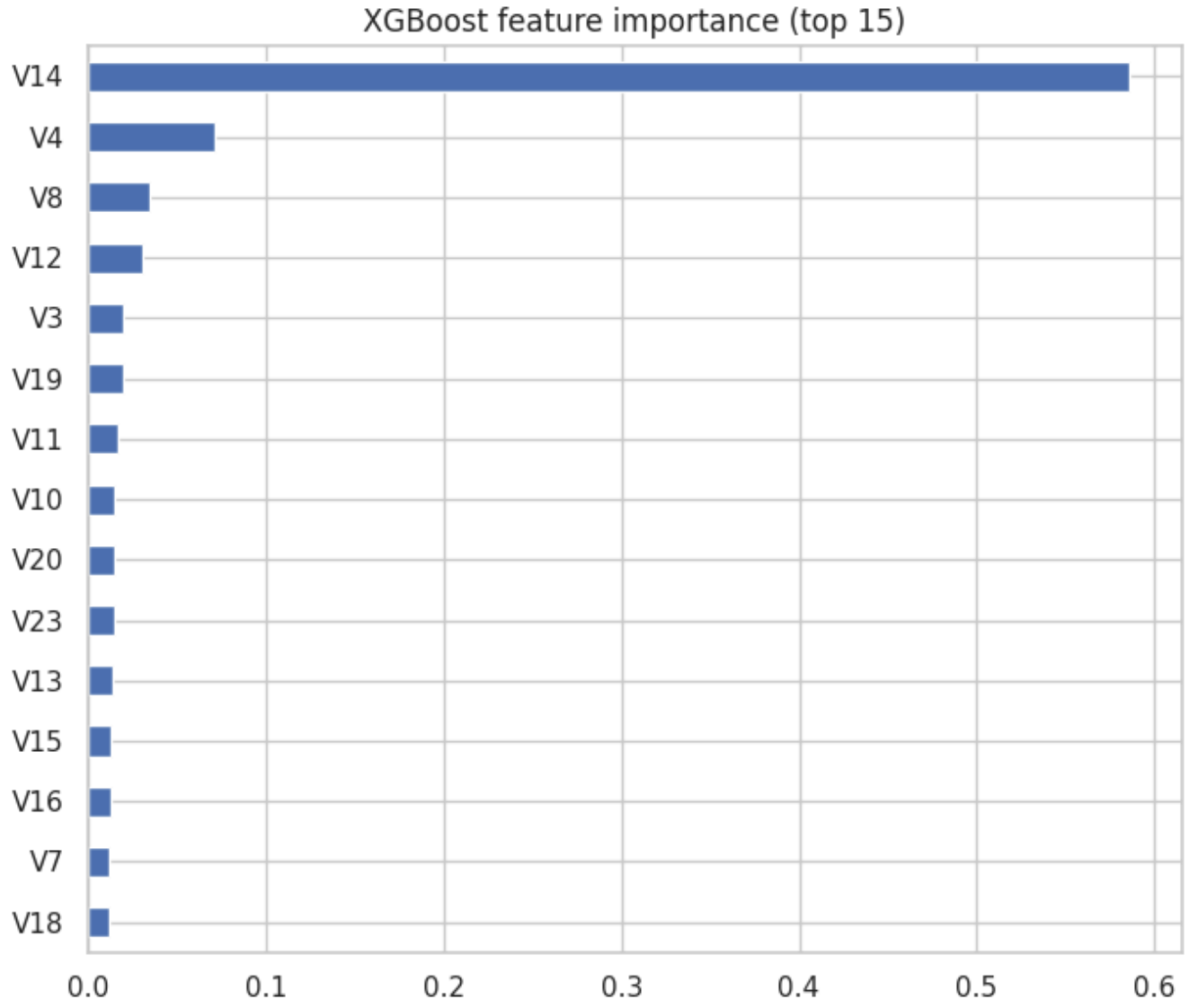
What this means for the business problem.

- **Logistic Regression is not usable as-is.** It catches the most fraud (recall 0.874) but with precision of about 0.05: for each fraud it catches it also flags about 17 legitimate transactions (1,414 / 83 and 1,482 / 83). A review team would drown in false alarms.
- **XGBoost is far more precise.** It misses 23 of 95 frauds (24%) but raises 1 false alarm in 56,651 legitimate transactions (0.002%).
- **SMOTE did not help here.** Logistic Regression with SMOTE has almost the same PR-AUC (0.676 vs 0.673) and slightly more false alarms than with class weights.
- **ROC-AUC hides the gap.** ROC-AUC is similar for Logistic Regression (0.966, 0.961) and XGBoost (0.976), yet PR-AUC differs a lot (about 0.67 vs 0.82). On data this imbalanced, PR-AUC is the more honest summary.

- **The threshold changed little.** Moving from the default 0.5 to 0.840 cut false alarms from 4 to 1 and cost one caught fraud (73 to 72). PR-AUC is identical (0.817) because the scores are the same; only the cut-off moved. With 95 frauds in the test set, one transaction is about 1 percentage point of recall, so this difference is within noise and should not be over-read.



The XGBoost curve stays high until recall reaches about 0.75, then drops sharply, while the Logistic Regression curves sit lower across most of the recall range.



Feature `V14` carries by far the most weight in the model (importance roughly 0.6, read from the chart), followed by `V4`; the rest each contribute little. Since `V14` is anonymised, I cannot explain what it represents.

Conclusion

Result. The tuned XGBoost with a threshold chosen on out-of-fold training predictions gives the best trade-off: 72 of 95 frauds caught, 1 false alarm, F1 0.857 and PR-AUC 0.817, against a PR-AUC of about 0.67 for Logistic Regression and about 0.00 for the Dummy baseline.

Limitations.

- The test set holds only 95 frauds, so every rate has wide uncertainty and small differences between models or thresholds may be noise. I ran one train/test split and did not repeat it.

- The data covers about two days and is split at random. A real system would be trained on the past and tested on the future, and fraud patterns change over time.
- Features are anonymised, so the model cannot be explained in business terms.
- The tuning search tried only 10 parameter sets, and the threshold maximises F1 rather than real costs.
- I did not estimate money saved, because that needs the actual losses from missed fraud and the cost of reviewing a false alarm.

Future work. Choose the threshold from real fraud losses and review costs; evaluate with a time-based split and repeated cross-validation; widen the hyperparameter search and compare other models; and deploy the best model behind a FastAPI endpoint for live scoring.